

Rome: Rise Over My Essence

(Eleve-se sobre minha essência)

Gustavo Lucena Aguiar¹, Kayron Gabriel Gomes Nascimento ², Lucas Alves Santana³, Matheus Souza Ferreira⁴.

¹Graduando do curso de Engenharia de Software, Universidade Católica de Brasília-DF.

²Graduando do curso de Engenharia de Software, Universidade Católica de Brasília-DF.

³Graduando do curso de Engenharia de Software, Universidade Católica de Brasília-DF.

⁴Graduando do curso de Engenharia de Software, Universidade Católica de Brasília-DF.

RESUMO

Este artigo apresenta o desenvolvimento da Rome: *Rise Over My Essence*, um e-commerce nacional e de autoria própria voltado para a comercialização de roupas masculinas e femininas, construído com base em princípios avançados de User Experience (UX), User Interface (UI) e diretrizes internacionais de usabilidade. A plataforma segue uma arquitetura completa composta por página inicial, categorias, página de produto, carrinho, checkout, fidelização, login, cadastro e gateway de pagamento, articulada segundo boas práticas consolidadas e evidências empíricas. O frontend foi desenvolvido utilizando HTML e CSS, em conjunto com padrões de UX e UI para garantir clareza, consistência e acessibilidade visual. No backend, empregou-se Java, aliado a uma camada de persistência híbrida composta por bancos SQL e NoSQL, permitindo robustez, escalabilidade e integridade dos dados.

Segundo o Baymard Institute (2023), “a maior parte das perdas de conversão é causada por fricções evitáveis na interface”, o que orientou decisões estruturais no design da Rome. Da mesma forma, as heurísticas de usabilidade do Nielsen Norman Group contribuíram para definir previsibilidade e fluidez na jornada digital. A identidade visual baseada na psicologia das cores em preto, branco e detalhes em laranja que reforçam à robustez, modernidade, elegância e transparência, refletindo a essência conceitual da marca. Assim, a Rome demonstra como fundamentos de UX, UI e engenharia de software podem ser aplicados à construção de um e-commerce autoral, competitivo e alinhado às expectativas do mercado contemporâneo.

Palavras-chaves: E-commerce; HTML; Identidade Visual; Java; NoSQL; Plataforma Rome; SQL; User Experience; User Interface; Usabilidade.

ABSTRACT

This article presents the development of **Rome: Rise Over My Essence**, a national and independently created e-commerce platform focused on the sale of men's and women's clothing, built upon advanced principles of User Experience (UX), User Interface (UI), and international usability guidelines. The platform is structured around a complete digital shopping architecture composed of a homepage, categories, product page, cart, checkout, loyalty features, login, registration, and payment gateway—organized according to consolidated best practices and empirical evidence. The frontend was developed using **HTML and CSS**, combined with UX and UI standards to ensure clarity, consistency, and visual accessibility. On the backend, **Java** was employed, supported by a hybrid data layer using both **SQL and NoSQL** databases, providing robustness, scalability, and integrity to the system.

According to the Baymard Institute (2023), “most conversion losses are caused by avoidable interface frictions,” which guided structural design decisions throughout the development of Rome. Additionally, usability heuristics from the Nielsen Norman Group helped define predictability and fluidity within the digital journey. The visual identity, based on the psychology of the colors black, white, and orange—representing robustness, modernity, elegance, and transparency—reinforces the conceptual

essence of the brand. Thus, Rome demonstrates how UX, UI, and software engineering principles can be effectively applied to the creation of an original, competitive e-commerce platform aligned with the expectations of the contemporary fashion market.

Keywords: CSS; E-commerce; HTML; Java; NoSQL; Rome Platform; SQL; Usability; User Experience; User Interface; Visual Identity.

1. INTRODUÇÃO

Nesse contexto, o e-commerce Rome: Rise Over My Essence foi desenvolvido como uma plataforma nacional e de autoria própria, especializada na venda de roupas masculinas e femininas. Sua construção fundamenta-se em princípios avançados de UX e UI, combinados a uma arquitetura tecnológica robusta composta por frontend em HTML e CSS e backend em Java, apoiado por um sistema de persistência de dados híbrido que integra bancos SQL e NoSQL. Essa combinação permite estabilidade, escalabilidade, integridade e velocidade na manipulação de dados, favorecendo tanto o desempenho do sistema quanto a fluidez da experiência do usuário.

A jornada de navegação da Rome foi estruturada segundo boas práticas consolidadas do mercado internacional, contemplando página inicial, categorias, página de produto, carrinho, checkout, fidelização, login, cadastro e gateway de pagamento. O *Google UX Playbook for E-commerce* reforça que “experiências otimizadas para dispositivos móveis aumentam consideravelmente a taxa de conversão” (GOOGLE, 2023), orientando a construção de uma interface responsiva e acessível. Complementarmente, a Shopify destaca que “o design consistente e o fluxo de compra simplificado potencializam os resultados de conversão em lojas de moda” (SHOPIFY, 2023), embasando escolhas estruturais adotadas no projeto.

A identidade visual da Rome é baseada na psicologia das cores preto, branco e detalhes em laranja e foi construída e pensada para transmitir robustez, modernidade, elegância e transparência, alinhando estética e significado à essência da marca. Essa abordagem humanizada permite que o usuário se conecte

intuitivamente com a proposta do e-commerce, reforçando a mensagem central da marca: elevar a individualidade do consumidor por meio de uma experiência digital limpa, segura e agradável.

Assim, a Rome se estabelece como um projeto que integra rigor técnico, fundamentos de UX/UI e sensibilidade estética, demonstrando como a união entre tecnologia, design e comportamento humano pode resultar em um e-commerce autoral, competitivo e alinhado às exigências do mercado contemporâneo.

2. OBJETIVOS

Desenvolver o e-commerce Rome: Rise Over My Essence, uma plataforma nacional e autoral de moda masculina e feminina, integrando princípios avançados de UX e UI a uma arquitetura tecnológica composta por frontend em HTML, CSS e backend em Java com persistência híbrida em SQL e NoSQL. O objetivo central é criar uma experiência de compra fluida, intuitiva e visualmente coerente com a identidade da marca, garantindo desempenho, segurança e facilidade de navegação em todas as etapas da jornada do usuário.

1. **Estruturar a Página Inicial** de maneira clara e orientada às necessidades do usuário, utilizando princípios de UX e UI para facilitar o acesso às categorias e produtos.
2. **Organizar categorias intuitivas**, reduzindo esforço cognitivo do usuário, conforme o Nielsen Norman Group, que destaca que “a previsibilidade é fundamental para orientar o consumidor” (NIELSEN; NORMAN, 2022).
3. **Desenvolver páginas de produto informativas e visuais**, com HTML e CSS otimizados, garantindo legibilidade, clareza e estímulo à decisão.
4. **Construir um carrinho funcional e objetivo**, permitindo revisão rápida e continuidade natural da compra.
5. **Implementar um checkout eficiente e seguro**, reduzindo etapas desnecessárias e garantindo fluidez no processo de pagamento.
6. **Criar fluxos de login e cadastro simplificados**, evitando interrupções e mantendo aderência às melhores práticas de usabilidade.
7. **Integrar mecanismos de fidelização**, como contas personalizadas e recomendações de produtos.

8. **Empregar SQL e NoSQL** de forma complementar, assegurando integridade, velocidade e escalabilidade nos processos de armazenamento e consulta.
9. **Realizar testes contínuos de usabilidade**, avaliando desempenho, compreensão e comportamento do usuário.

3. METODOLOGIA

3.1 Tipo de abordagem

Pesquisa aplicada, com abordagem mista: quantitativa na definição de chaves, restrições, índices e testes de consistência; qualitativa na interpretação dos requisitos do domínio de e-commerce e na validação funcional das operações CRUD.

3.2 Materiais e ferramentas

brModelo (modelagem conceitual e lógica), MySQL/Workbench ou CLI (implementação física), MongoDB/mongosh (camada NoSQL para eventos e cache), Java/Maven (protótipo da aplicação), padrão DAO para acesso a dados (ex.: AvaliacaoDAO, ProdutoDAO, PedidoDAO).

3.3 Etapas do procedimento

1. Levantamento de requisitos: identificação das entidades centrais (Cliente, Produto, Fornecedor, Pedido, Pagamento, Avaliação) e de seus relacionamentos.
2. Modelagem conceitual: elaboração do DER no brModelo, contemplando cardinalidades e integrações.
3. Normalização: refinamento até a 3ª Forma Normal, reduzindo redundâncias e anomalias de atualização.
4. Modelo lógico e físico: tradução do conceitual para tabelas, atributos, PKs/FKs e geração do script Ecommerce_BD.sql com restrições e índices.
5. Implementação Java: criação dos DAOs para encapsular operações CRUD e reutilizar conexões (AvaliacaoDAO.java, ProdutoDAO.java, PedidoDAO.java, entre outros).

6. Extensão NoSQL: execução de `mongodb_setup.js` para criar a base `EcommerceRealtime`, coleções de eventos e cache de catálogo, com pipelines de agregação exemplificadas.
7. Testes e validação: inserções, consultas e alterações nas entidades principais; verificação de integridade referencial no MySQL; conferência de pipelines e índices no MongoDB; checagem de coerência entre camadas (modelo → script SQL → DAOs → consultas/resultados).

3.4 Critérios de aceitação

1. Todas as entidades e relacionamentos previstos criados conforme `Ecommerce_BD.sql`.
2. Integridade referencial preservada (FKs sem violação; remoções e alterações consistentemente propagadas).
3. Operações CRUD funcionais via camada DAO, retornando objetos completos e normalizados.
4. Pipelines Mongo operando sobre eventos e cache, com índices aplicados conforme o script.
5. Documentação e scripts reproduzíveis em ambiente acadêmico (MySQL + MongoDB).

3.5 Limitações

O escopo de testes é básico (funcionais e de consistência); não cobre carga, concorrência ou recuperação em falhas. A integração NoSQL é demonstrativa e não aborda replicação ou sharding.

4. DESCRIÇÃO DOS SISTEMAS

O sistema de e-commerce proposto integra cadastro e gestão de clientes (PF/PJ), fornecedores e produtos, emissão de pedidos e pagamentos, além de avaliações de produtos. A solução combina persistência relacional em MySQL para garantir integridade transacional e uma camada NoSQL em MongoDB para armazenar eventos e cache de catálogo, favorecendo consultas ágeis e análises pontuais.

Para consulta das implementações, documentação complementar e acesso integral ao código-fonte, pode-se verificar as referências do devido artigo, onde tem todas as informações adicionais e o link do Github que estão devidamente organizados e disponibilizados.

4.1 Funcionalidades

1. Cadastro, atualização e consulta de clientes (PF/PJ), fornecedores e produtos.
2. Emissão e acompanhamento de pedidos com itens, cálculo de totais e vínculo com pagamentos.
3. Registro de avaliações (nota e comentário) por cliente e produto, com consultas por item.
4. Geração e manutenção de notas fiscais e telefones associados.
5. Operações CRUD encapsuladas via DAOs em Java, preservando validação e consistência dos dados.
6. Camada complementar em MongoDB para eventos de pedidos e cache de catálogo, apoiando leitura rápida e análises exploratórias.

4.2 Tecnologias Utilizadas

1. Frontend: páginas de templates estáticos (ex.: Thymeleaf/HTML) para ilustração de fluxo e interação básica.
2. Backend: Java (Maven), padrão DAO para persistência (AvaliacaoDAO, ProdutoDAO, PedidoDAO), conectando-se ao SGBD relacional.
3. SGBD relacional: MySQL (script Ecommerce_BD.sql, chaves primárias e estrangeiras, índices, constraints).
4. SGBD NoSQL: MongoDB (script mongodb_setup.js) para coleções de eventos e cache de catálogo; uso de mongosh para exploração e pipelines de agregação.
5. Ferramentas de apoio: brModelo para DER conceitual/lógico, MySQL Workbench/CLI, mongosh.

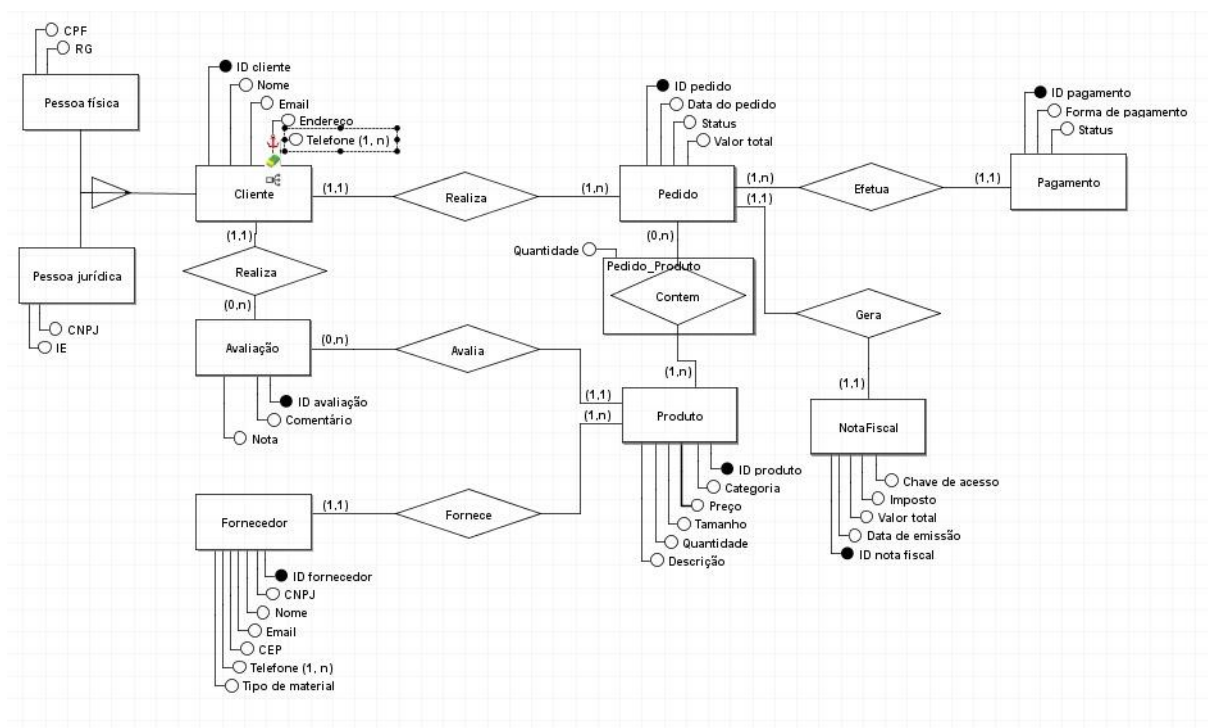
4.3 Justificativa de Cada Escolha Tecnológica

1. MySQL (relacional): amplamente difundido, sólido em integridade referencial, suporte a 3FN e ótimo para transações de pedidos e pagamentos. Alinha-se a cenários acadêmicos e profissionais, com tooling acessível (Workbench/CLI).
2. Java + DAO: Na arquitetura MVC, sem o “V” de View já que o sistema utiliza HTML, ele promove separação de camadas, reutilização de conexão e encapsulamento de regras de negócio, facilitando manutenção e testes unitários/funcionais. Maven garante gestão padronizada de dependências e build.
3. MongoDB (NoSQL): adequado para dados semiestruturados (eventos, cache de catálogo) e consultas analíticas rápidas via pipelines de agregação, sem sobrecarregar o banco transacional.
4. brModelo: ferramenta acadêmica clara para elaborar DER e guiar a passagem conceitual → lógico → físico.
5. Templates HTML/Thymeleaf: permitem validar fluxo de interface e navegação, mesmo em caráter demonstrativo, reduzindo atrito de configuração de frontends mais pesados.
6. Essa combinação tecnológica equilibra integridade transacional (MySQL) com agilidade em leitura e experimentação analítica (MongoDB), enquanto o backend em Java com DAOs assegura organização do código e portabilidade para ambientes de ensino e prototipagem.

5. MODELAGEM DO BANCO DE DADOS

5.1 DER: DIAGRAMA ENTIDADE-RELACIONAMENTO

Imagem 1 - DER: Diagrama Entidade-Relacionamento



Fonte: Diagrama realizado pelo autor.

5.2 Explicação das entidades

O Diagrama de Entidade-Relacionamento (DER) apresentado modela um sistema de comércio eletrônico (e-commerce), detalhando as principais entidades envolvidas e como elas se interconectam. A seguir, apresentamos a descrição de cada entidade, seus atributos e os relacionamentos estabelecidos.

5.2.1. Entidades e Atributos

Tabela 1 – Entidades e Atributos

Entidade	Chave Primária	Atributos	Descrição
Cliente	ID cliente	Nome, Email, Endereço (Composto), Telefone (Multivalorado)	Representa o comprador no sistema. Possui atributos básicos de contato e um identificador único. O atributo Endereço é composto (rua, número, bairro etc.), e Telefone é

			multivalorado, permitindo múltiplos contatos.
Pessoa física	(Herdada)	CPF, RG	Subtipo de Cliente para pessoas físicas, com atributos específicos de identificação.
Pessoa jurídica	(Herdada)	CNPJ, IE	Subtipo de Cliente para empresas, com atributos específicos de identificação fiscal.
Pedido	ID pedido	Data do pedido, Status, Valor total	Representa uma transação de compra realizada por um cliente. Contém informações sobre a data, o estado atual do pedido e o valor total da compra.
Produto	ID produto	Categoria, Preço, Tamanho, Quantidade, Descrição	Representa os itens disponíveis para venda no e-commerce. Possui um identificador único e detalhes como preço, estoque (Quantidade) e características.
Pagamento	ID pagamento	Forma de pagamento, Status	Registra os detalhes da transação financeira associada a um pedido.
NotaFiscal	ID nota fiscal	Chave de acesso, Imposto, Valor total, Data de emissão	Documento fiscal gerado para o pedido.

Avaliação	ID avaliação	Comentário, Nota	Representa o feedback de um cliente sobre um produto.
Fornecedor	ID fornecedor	CNPJ, Nome, Email, CEP, Telefone (Multivalorado), Tipo de material	Representa as empresas que fornecem os produtos. O atributo Telefone é multivalorado.

Fonte: Tabela realizada pelo autor.

5.2.2. Relacionamentos

Os relacionamentos definem as associações lógicas entre as entidades, com suas respectivas cardinalidades (participação mínima e máxima).

Tabela 2 - Relacionamentos

Relacionamento	Entidades Envolvidas	Cardinalidade	Descrição
Realiza	Cliente, Pedido	(1,1) Cliente - (1,n) Pedido	Um Cliente realiza um ou mais Pedidos. Cada Pedido é realizado por exatamente um Cliente.
Efetua	Pedido, Pagamento	(1,n) Pedido - (1,1) Pagamento	Um Pedido efetua exatamente um Pagamento. Um Pagamento está associado a um ou mais Pedidos (o que sugere que um pagamento pode cobrir múltiplos

			pedidos, embora a cardinalidade (1,1) no lado do Pagamento seja mais comum para um pagamento por pedido). Assumindo a modelagem, um pagamento é obrigatório para o pedido.
Contém	Pedido, Produto	(1,n) Pedido - (1,n) Produto	Relacionamento N:N (Muitos para Muitos) que é resolvido pela entidade associativa Pedido_Produto. Um Pedido contém um ou mais Produtos, e um Produto pode estar em um ou mais Pedidos.
Pedido_Produto	Pedido, Produto	(1,n) Pedido - (1,n) Produto	Entidade associativa do relacionamento Contém. Possui o atributo descritivo Quantidade, que indica quantos itens de um determinado

			Produto estão naquele Pedido.
Gera	Pedido, NotaFiscal	(1,1) Pedido - (1,1) NotaFiscal	Um Pedido gera exatamente uma NotaFiscal, e uma NotaFiscal é gerada por exatamente um Pedido.
Realiza	Cliente, Avaliação	(1,1) Cliente - (0,n) Avaliação	Um Cliente realiza zero ou mais Avaliações. Cada Avaliação é realizada por exatamente um Cliente.
Avalia	Avaliação, Produto	(0,n) Avaliação - (1,1) Produto	Uma Avaliação avalia exatamente um Produto. Um Produto pode ter zero ou mais Avaliações.
Fornece	Fornecedor, Produto	(1,1) Fornecedor - (1,n) Produto	Um Fornecedor fornece um ou mais Produtos. Cada Produto é fornecido por exatamente um Fornecedor.
Especialização/Generalização	Cliente, Pessoa física, Pessoa jurídica	(Total Disjunta) e	A entidade Cliente é generalizada em dois subtipos: Pessoa física e Pessoa jurídica. A

			participação é total (todo cliente é um ou outro) e disjunta (um cliente não pode ser ambos ao mesmo tempo).
--	--	--	--

Fonte: Tabela realizada pelo autor.

5.3 Justificativa de índices, triggers, procedures/functions e entre outros

O Diagrama de Entidade-Relacionamento (DER) foi usado para fazer o arquivo Ecommerce_BD.sql e a partir dele conseguimos descrever:

1. Índices e chaves: aplicar PK/FK indexadas e únicas como as definidas para fornecedores (uq_fornecedor_cnpj, uq_fornecedor_email em Ecommerce_BD.sql (linhas 12-13)) e clientes (uq_cliente_email em Ecommerce_BD.sql (linhas 59)) garante unicidade em documentos/e-mails e acelera joins críticos (ex.: pedido↔cliente, pedido_produto↔produto, avaliacao↔cliente/produto, Ecommerce_BD.sql (linhas 73-79, 99-103, 142-146, 164-167)). Checks complementam a integridade de domínio (ex.: CEP/CNPJ em Ecommerce_BD.sql (linhas 15-16), preços/prazos não negativos em Ecommerce_BD.sql (linhas 80-81), impostos/valores em Ecommerce_BD.sql (linhas 126-128), notas de avaliação entre 0 e 5 em Ecommerce_BD.sql (linhas 169)), prevenindo dados inválidos na origem e reduzindo custo de correção na aplicação.
2. Triggers e funções de ID: os gatilhos trg_cliente_define_id, trg_fornecedor_define_id, trg_pedido_define_id, trg_pagamento_define_id e trg_usuario_define_id (faixa Ecommerce_BD.sql (linhas 270-320)) automatizam a geração de identificadores via fn_gerar_id_pedido, fn_gerar_id_pagamento, fn_gerar_id_usuario (Ecommerce_BD.sql: linhas 230-260), evitando colisões e entradas sem chave; o uso de UUID_SHORT() também minimiza predição de IDs.
3. Trigger de consistência de totais: trg_pedido_produto_recalcula_total e trg_pedido_produto_recalcula_total_delete (Ecommerce_BD.sql: linhas 330-

- 350) recalculam Valor_total com fn_calcular_total_pedido (Ecommerce_BD.sql: linhas 240-248) a cada inserção/remoção de item, garantindo que o total do pedido permaneça sincronizado sem depender da camada de aplicação.
4. Views: vw_resumo_pedidos e vw_monitoramento_estoque (Ecommerce_BD.sql: linhas 360-375) oferecem acesso seguro e simplificado a relatórios recorrentes (totais e status de pedidos; níveis de estoque), evitando repetição de joins e regras de categorização na aplicação.
 5. Procedures: sp_criar_pagamento e sp_atualizar_status_pedido (Ecommerce_BD.sql: linhas 380-400) encapsulam operações sensíveis — criação de pagamento e vínculo ao pedido; atualização de status restrita aos valores do domínio — padronizando fluxos de negócio e reduzindo duplicação de SQL.
 6. Equilíbrio desempenho × integridade: optou-se por índices no mínimo necessário (PK/FK e colunas de busca frequente) para preservar a performance de escrita em tabelas volumosas como pedido_produto e avaliacao. Triggers ficam focados em coerência derivada e geração de IDs, por serem baratos; lógica de negócio extensa é mantida em procedures ou na aplicação. Views e checks atuam como contratos de leitura e de domínio, reforçando qualidade dos dados e segurança operacional.

6. CONTROLE DE ACESSO

6.1 Descrição dos usuários e grupos

1. app_admin: perfil administrativo do esquema EcommerceBD, habilitado para SELECT, INSERT, UPDATE, DELETE, EXECUTE; designado para manutenção completa e execução de procedures/views.
2. app_analyst: perfil de leitura, autorizado apenas a SELECT em todo o esquema, adequado a consultas e relatórios.
3. app_integracao: perfil de integração externa com permissões restritas a produto (SELECT, INSERT, UPDATE) e pedido (SELECT, INSERT), orientado a ingestão controlada de catálogo e criação de pedidos.
4. root bloqueado (ALTER USER 'root'@'localhost' ACCOUNT LOCK), reforçando a política de privilégio mínimo e reduzindo a superfície de ataque.

6.2 Regras de acesso

1. Princípio do menor privilégio aplicado: cada usuário recebe apenas as operações necessárias ao seu papel.
2. Separação de funções: administração (app_admin), análise/BI (app_analyst) e integração (app_integracao) operam com escopos distintos, prevenindo elevação inadvertida de privilégios.
3. Execução de objetos armazenados: apenas app_admin possui EXECUTE, assegurando controle sobre procedures que alteram estado (sp_criar_pagamento, sp_atualizar_status_pedido).
4. Proteção de dados críticos: escrita em pedido limitada a integrações autorizadas; atualização de produtos controlada; leitura ampla somente ao perfil analista.
5. Hardening: desabilitar root limita vetores de acesso privilegiado; recomenda-se senha forte e isolamento de rede para cada conta, além de auditoria periódica de permissões.

7. USO DO BANCO NoSQL

7.1 Explicação técnica (MongoDB)

O MongoDB complementa o MySQL na base EcommerceRealtime, dedicado a dados semiestruturados e consultas rápidas. O script mongodb_setup.js cria pedido_eventos com validação via \$jsonSchema, garantindo campos essenciais (pedidoId, clienteId, tipoEvento, payload e dataEvento) e collation em português para ordenação/coerência lexical. Índices direcionados fortalecem desempenho: idx_pedido_data (pedidoId asc, dataEvento desc) acelera timelines por pedido; idx_cliente_tipo otimiza filtros por cliente e tipo de evento; e idx_texto_catalogo em catalogo_cache habilita busca textual com stemming em português, combinando descrição e categorias.

O uso de NumberLong alinha IDs ao domínio relacional, preservando integridade referencial lógica entre MySQL e MongoDB. Pipelines de agregação ilustram casos práticos: obter o status mais recente de cada pedido (ordenando eventos e agrupando por pedido) e montar vitrines por categoria a partir do cache de catálogo, reduzindo carga no MySQL transacional.

Dessa forma, eventos e cache ficam em uma camada flexível e performática, enquanto o núcleo transacional permanece estável no SGBD relacional.

7.2 Justificativa da aplicação no sistema

1. Camada de eventos (pedido_eventos): armazena histórico flexível (checkout, pagamento, logística) sem impactar o modelo relacional; o schema JSON preserva consistência mínima enquanto permite payload variável para contextos distintos. Índices temporais e por cliente/tipo aceleram consultas de monitoramento e auditoria.
2. Cache de catálogo (catalogo_cache): snapshots de produtos para leitura rápida em front-end, desacoplando consultas de catálogo do MySQL; o índice textual viabiliza busca por descrição/categoria sem sobrecarregar o SGBD transacional.
3. Pipelines demonstrativas: agregações para obter status atual por pedido e montagem de vitrines por categoria mostram como explorar dados semi-estruturados para UX e análises rápidas.
4. Integração híbrida: mantém o MySQL focado em transações e integridade (pedidos, pagamentos, estoque), enquanto o MongoDB absorve eventos e caches mutáveis, reduzindo contenção em cargas de leitura/escrita variáveis e facilitando evolução de schema de eventos.

8. CONCLUSÃO

A solução proposta entrega um modelo relacional em MySQL normalizado até a 3ª Forma Normal, reforçado por constraints, chaves e gatilhos que asseguram integridade transacional em entidades críticas (pedido, pagamento, itens, avaliação). A camada Java, estruturada em DAOs, demonstra boas práticas de separação de responsabilidades e validação, enquanto procedures, views e funções encapsulam operações sensíveis e consultas recorrentes no SGBD.

Em paralelo, o módulo MongoDB estende o ecossistema para dados semiestruturados, tratando eventos de pedidos e cache de catálogo com índices e pipelines otimizados para leitura e análise rápida, aliviando o banco transacional. Essa arquitetura híbrida evidencia a complementaridade entre modelos relacional e NoSQL:

integridade e consistência no core transacional, flexibilidade e agilidade na camada de eventos e cache.

Como passos futuros, destacam-se testes de carga e concorrência, monitoração de índices, ajustes finos de perfis de acesso e evolução das pipelines analíticas, consolidando o projeto como base sólida para um e-commerce escalável e auditável.

9. REFERÊNCIAS

BAYMARD INSTITUTE. *E-commerce UX Research*. 2023. Disponível em: <https://baymard.com>. Acesso em: 20 jan. 2025.

DEITEL, P. J.; DEITEL, H. M. *Java: Como Programar*. 10. ed. São Paulo: Pearson, 2016.

GOOGLE. *UX Playbook for E-commerce*. 2023. Disponível em: <https://playbook.thinkwithgoogle.com>. Acesso em: 20 jan. 2025.

HELMER, G. *SQL Essentials*. Rio de Janeiro: Brasport, 2020.

KAYRON007. *Projeto Banco de Dados E-commerce*. GitHub, 2025. Disponível em: https://github.com/Kayron007/projeto_banco_de_dados_ecommerce. Acesso em: 04 nov. 2025.

NIELSEN, J.; NORMAN, D. *Nielsen Norman Group – UX Research & Principles*. 2022. Disponível em: <https://www.nngroup.com>. Acesso em: 04 nov. 2025.

SHOPIFY. *E-commerce Design Guidelines*. 2023. Disponível em: <https://www.shopify.com/partners/blog>. Acesso em: 04 nov. 2025.

SILVA, F. R.; MARTINS, L. C. *NoSQL: Modelos, Armazenamento e Consultas*. Porto Alegre: Bookman, 2021.